These two lines compile.
They still crash.

```
class Numbers {
    int* data;

public:
    Numbers() : data(new int[3]{}) {}
    ~Numbers() { delete[] data; }
};

int main() {
    Numbers a;
    Numbers b = a;
}
```

Both objects free the same memory.

swipe →

Two kinds of copy

YOU WRITE	WHAT RUNS	ITS JOB
`Numbers b = a;`	copy constructor	build a **new** object from `a`
`b = a;`	copy assignment	overwrite an **existing** `b` with `a`

If you write neither, the compiler writes both for you. It copies every member, one by one.

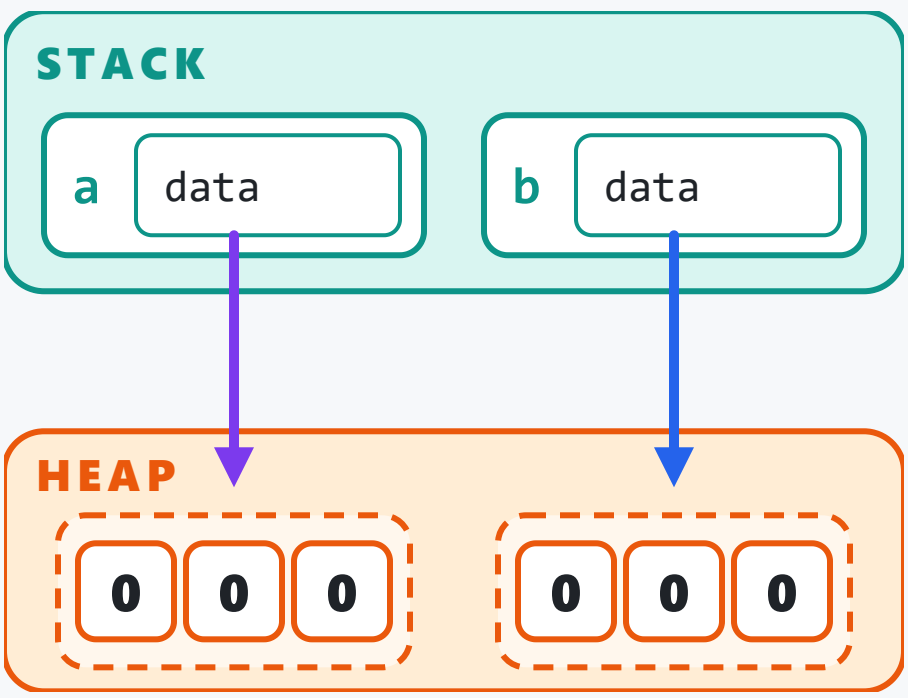
For an `int` or a `std::string` member, that is exactly right.

For a raw pointer it copies the address, not what it points to.

What the copy really does

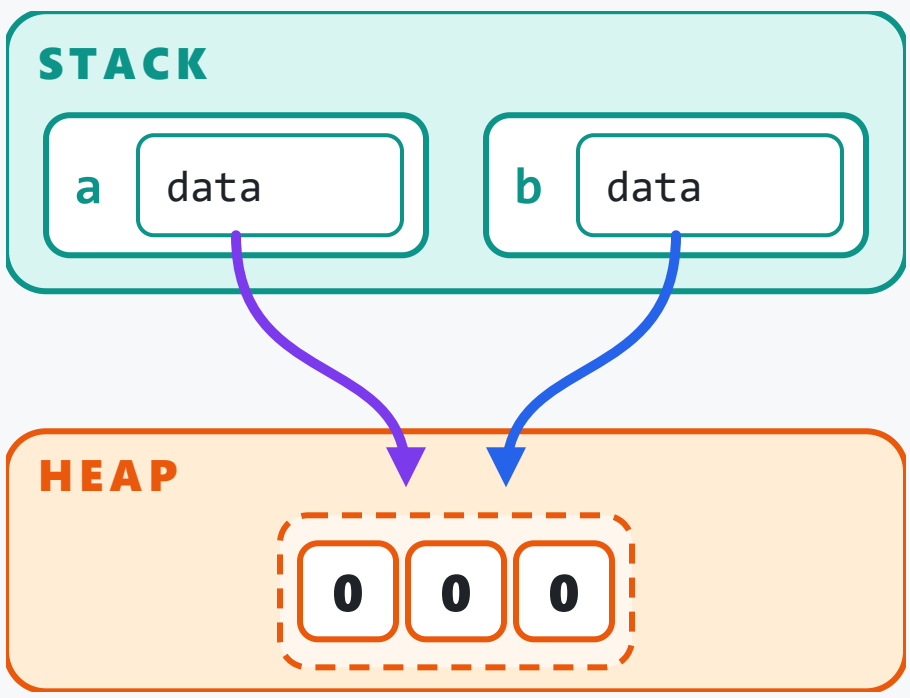
```
Numbers a;
Numbers b = a;
```

What you want



Two blocks, one each.

What you get



Two pointers, one block.

The compiler copied `data`, not the three ints.

When both objects die, the block is **freed twice**.

Then the scope ends

```
}   // b is destroyed, then a
```

STEP	WHAT RUNS	RESULT
1	`~b`: `delete[] data`	the block is freed
2	`~a`: `delete[] data`	**the same block, freed again**

Double delete: undefined behaviour.

I ran this exact class: on my machine it crashed with a heap-corruption error.

- changing `b`'s ints also changes `a`'s
- destroy one, and the other points at freed memory

The Rule of Three

If you write one of these, you almost certainly need all three.

YOU WRITE	BECAUSE
`~Numbers()` destructor	the class owns a resource and must release it
`Numbers(const Numbers&)` copy constructor	a copy must not share the resource
`operator=(const Numbers&)` copy assignment	the same for `b = a`: copy into `b`, never share

No compiler enforces it. It is a design rule.

The fix: copy the ints

```
Numbers(const Numbers& o)
    : data(new int[3]) {
    std::copy_n(o.data, 3, data);
}
```

New block, same values.

`a` and `b` now own **separate** blocks.

`operator=` needs the same fix. Together with the destructor, that is the Rule of Three.

Or write none of them

```
struct Numbers {
    std::vector<int> data = {0, 0, 0};
};
```

`std::vector` already copies and frees correctly.

That is the **Rule of Zero**.

Rule of Zero or Rule of Three: which do you reach for?